

3.5 🏗️ P2 里程碑1：移动机器人 URDF

所属阶段： 第三阶段 · 坐标变换与机器人建模

讲次数量： 4 节

预计时长： ~40 分钟

前置依赖： 3.3 Xacro 进阶建模、3.4 仿真准备与机器人描述包

学习目标

完成本节后，学员将能够：

- 为差分驱动移动机器人添加驱动轮、万向轮和激光雷达，构建完整的 TF 树
- 独立完成一个参考 TurtleBot3 Burger 的完整 URDF/Xacro 模型
- 使用 `check_urdf`、`view_frames`、RViz 2 全面验证模型正确性
- 交付符合项目二里程碑 1 要求的 `my_robot_description` 功能包

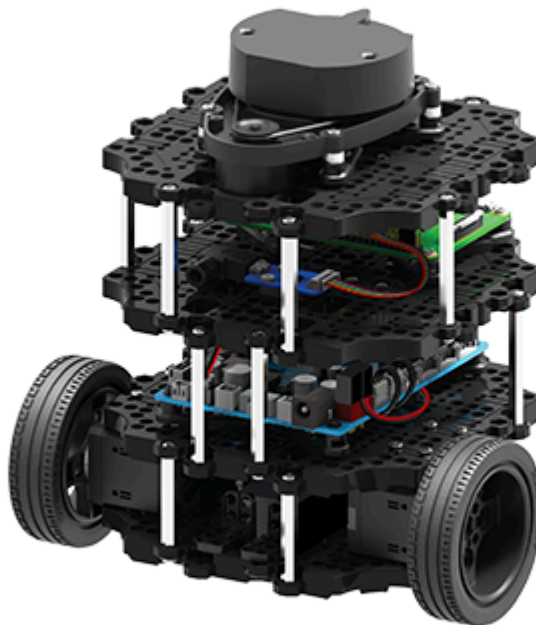
讲次 1：为移动机器人添加轮子

知识点

3.3 ~ 3.4 中我们已具备 Xacro 模块化、物理标签和 Launch 可视化的基础。本节在此基础上，为移动机器人添加**驱动轮**和**后万向轮 (caster)**，构成可运动的差速驱动底盘。

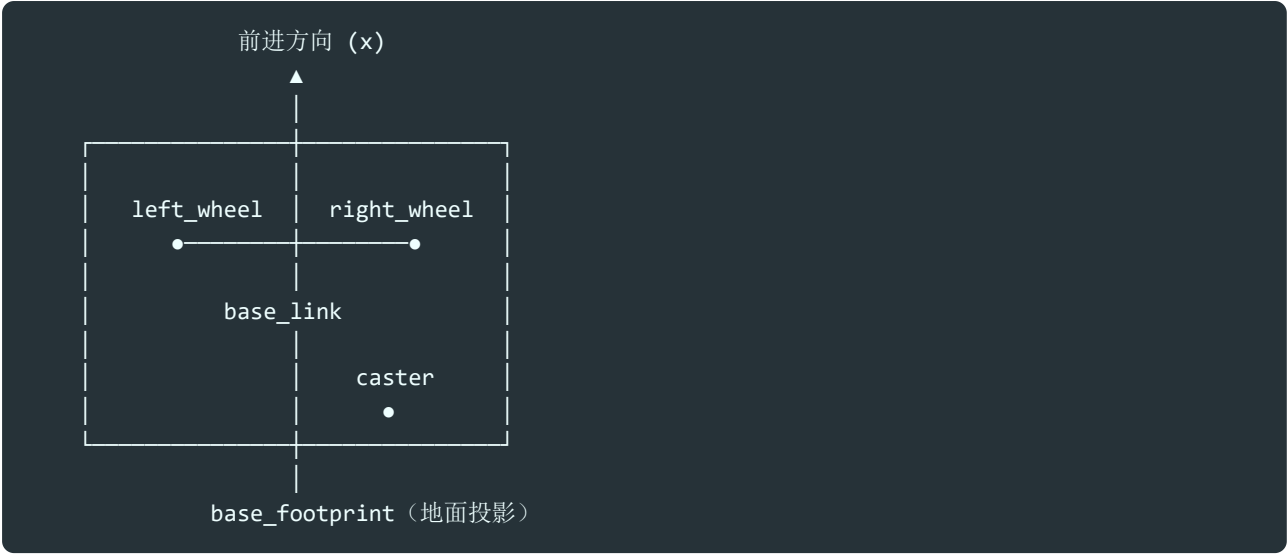
我们的 URDF 模型参考 TurtleBot3 Burger 的硬件尺寸：

TURTLEBOT3 Burger



Raspberry Pi[®] 4 (4GB)

差速驱动运动学模型



参数	TurtleBot3 Burger 值	说明
轮半径 (wheel_radius)	0.033 m	直径 66 mm
轮宽 (wheel_width)	0.018 m	轮厚度
轮间距 (wheel_separation)	0.160 m	左右轮中心距
轮中心到车体中心线 (wheel_ygap)	0.080 m	= separation / 2
万向轮位置	x=-0.081, z=-0.004	底盘后方

更新 properties.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- 底盘 -->
  <xacro:property name="base_length" value="0.178"/>
  <xacro:property name="base_width" value="0.138"/>
  <xacro:property name="base_height" value="0.109"/>
  <xacro:property name="base_mass" value="0.826"/>

  <!-- 轮子 -->
  <xacro:property name="wheel_radius" value="0.033"/>
  <xacro:property name="wheel_width" value="0.018"/>
  <xacro:property name="wheel_mass" value="0.0285"/>
  <xacro:property name="wheel_ygap" value="0.080"/>
  <xacro:property name="wheel_separation" value="0.160"/>

  <!-- 万向轮 -->
  <xacro:property name="caster_radius" value="0.005"/>
  <xacro:property name="caster_mass" value="0.005"/>
  <xacro:property name="caster_x" value="-0.081"/>
  <xacro:property name="caster_z" value="-0.004"/>

  <!-- 激光雷达 -->
  <xacro:property name="scan_radius" value="0.050"/>
  <xacro:property name="scan_height" value="0.040"/>
  <xacro:property name="scan_mass" value="0.114"/>
  <xacro:property name="scan_z" value="${base_height + scan_height / 2.0}"/>

</robot>
```

更新 macros.xacro——驱动轮宏

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- 惯性宏由主文件 my_robot.urdf.xacro 统一 include, 此处直接使用 -->

  <!-- 驱动轮: continuous 关节, 可无限旋转 -->
  <xacro:macro name="drive_wheel" params="prefix y_reflect">
    <link name="${prefix}_wheel_link">
      <inertial>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <mass value="${wheel_mass}"/>
        <xacro:inertia_cylinder_y
          mass="${wheel_mass}"
          radius="${wheel_radius}"
          length="${wheel_width}"/>
      </inertial>
      <visual>
        <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
        </geometry>
        <material name="black"/>
      </visual>
      <collision>
        <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
        </geometry>
      </collision>
    </link>

    <joint name="${prefix}_wheel_joint" type="continuous">
      <parent link="base_link"/>
      <child link="${prefix}_wheel_link"/>
      <origin xyz="0 ${y_reflect * wheel_ygap} 0" rpy="0 0 0"/>
      <axis xyz="0 1 0"/>
    </joint>
  </xacro:macro>

  <!-- 万向轮: fixed 关节, 被动支撑 -->
  <xacro:macro name="caster_wheel" params="prefix x_pos z_pos">
    <link name="${prefix}_caster_link">
      <inertial>
        <origin xyz="0 0 0" rpy="0 0 0"/>
        <mass value="${caster_mass}"/>
        <inertia ixx="1e-5" ixy="0" ixz="0"
          iyy="1e-5" iyz="0" izz="1e-5"/>
      </inertial>
      <visual>
        <geometry>
          <sphere radius="${caster_radius}"/>
        </geometry>
        <material name="black"/>
      </visual>
    </link>
  </xacro:macro>
</robot>
```

```

    <collision>
      <geometry>
        <sphere radius="${caster_radius}"/>
      </geometry>
    </collision>
  </link>

  <joint name="${prefix}_caster_joint" type="fixed">
    <parent link="base_link"/>
    <child link="${prefix}_caster_link"/>
    <origin xyz="${x_pos} 0 ${z_pos}" rpy="0 0 0"/>
  </joint>
</xacro:macro>

</robot>

```

关节类型选择

关节	类型	原因
left/right_wheel_joint	continuous	驱动轮可无限旋转，需要角速度控制
back_caster_joint	fixed	万向轮被动跟随，仿真中可后期改为 continuous
base_joint	fixed	base_footprint 到 base_link 的固定偏移
scan_joint	fixed	激光雷达固定安装

continuous 关节的 axis

```
<axis xyz="0 1 0"/>
```

表示轮子绕 **y 轴** 旋转。当收到正角速度时，机器人前进。这与差速驱动的标准约定一致。

在主文件中调用

```

<!-- 驱动轮 -->
<xacro:drive_wheel prefix="left" y_reflect="1"/>
<xacro:drive_wheel prefix="right" y_reflect="-1"/>

<!-- 后万向轮 -->
<xacro:caster_wheel prefix="back" x_pos="${caster_x}" z_pos="${caster_z}"/>

```

验证轮子关节

```
xacro my_robot.urdf.xacro | grep -E 'joint name|type='
```

预期输出：

```
joint name="base_joint" type="fixed"
joint name="scan_joint" type="fixed"
joint name="left_wheel_joint" type="continuous"
joint name="right_wheel_joint" type="continuous"
joint name="back_caster_joint" type="fixed"
```

```
ros2 launch my_robot_description display.launch.py
# 在 joint_state_publisher_gui 中拖动 left/right_wheel_joint
# 预期：轮子旋转，RViz 中模型实时更新
```

关键点

- 差速驱动 = 两个 continuous 驱动轮 + 至少一个支撑轮 (caster)
 - 驱动轮 joint 的 origin $y = \pm \text{wheel_ygap}$, $z = 0$ (轮心在 base_link 底部)
 - continuous 关节 axis = `0 1 0`, 绕 y 轴旋转
 - 万向轮用 sphere + fixed joint, 质量极小但惯性不能为零
 - TurtleBot3 Burger 轮间距 0.160 m、轮半径 0.033 m 是仿真验证过的参数
-

讲次 2：【Activity】完成移动机器人的完整 URDF

知识点

项目目标

综合运用 3.1 ~ 3.4 和本讲第 1 节的知识，在 `my_robot_description` 功能包中交付一个完整的差速驱动移动机器人 Xacro 模型。

交付要求

#	要求	验证方式
1	包含 6 个 link	<code>check_urdf</code>
2	包含 5 个 joint	<code>urdf_to_graphviz</code>
3	5 个实体 link 均有 visual + collision + inertial	<code>grep</code> 计数
4	使用 Xacro 模块化 (≥5 个 xacro 子文件 + 1 个主文件)	检查文件结构
5	配置 Gazebo 材质颜色	检查 <code>gazebo_materials.xacro</code>
6	Launch 文件可一键启动 RViz 显示	<code>ros2 launch</code>
7	TF 树以 <code>base_footprint</code> 为根	<code>view_frames</code>

必须包含的 Link 和 Joint

```
base_footprint
├── [base_joint, fixed]
│   └── base_link
│       ├── [scan_joint, fixed] → base_scan
│       ├── [left_wheel_joint, continuous] → left_wheel_link
│       ├── [right_wheel_joint, continuous] → right_wheel_link
│       └── [back_caster_joint, fixed] → back_caster_link
```

完整主文件模板

```
urdf/my_robot.urdf.xacro
```



```
<?xml version="1.0"?>
<robot name="my_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:include filename="my_robot/properties.xacro"/>
  <xacro:include filename="my_robot/materials.xacro"/>
  <xacro:include filename="my_robot/inertia_macros.xacro"/>
  <xacro:include filename="my_robot/macros.xacro"/>
  <xacro:include filename="my_robot/base.xacro"/>
  <xacro:include filename="my_robot/sensors.xacro"/>
  <xacro:include filename="my_robot/wheels.xacro"/>
  <xacro:include filename="my_robot/gazebo_materials.xacro"/>

</robot>
```

这个主文件只负责把各个子模块拼装起来，真实的 `base_link`、`base_scan` 和轮组定义分别放在 `base.xacro`、`sensors.xacro`、`wheels.xacro` 中。这样后续进入 Gazebo 仿真阶段时，底盘、传感器和插件配置可以继续独立演进。

学员任务步骤

1. 按 3.4 讲次 7 创建 `my_robot_description` 功能包（如尚未创建）
2. 按模板创建所有 xacro 子模块文件
3. 添加 `inertia_macros.xacro`（从 3.4 讲次 4 复用）
4. 添加 `gazebo_materials.xacro`（从 3.4 讲次 6 复用）
5. 创建 `display.launch.py` 和 `my_robot.rviz`
6. `colcon build` 并启动验证
7. 运行自检清单（见讲次 4）

自检命令

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash

# 结构检查
xacro $(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/my_robot.urdf.xacro
  -o /tmp/my_robot.urdf
check_urdf /tmp/my_robot.urdf

# 可视化
ros2 launch my_robot_description display.launch.py
```

关键点

- 里程碑 1 交付物是完整可运行的 `my_robot_description` 功能包
- 6 个 link、5 个 joint 构成标准差速驱动 TF 树
- 所有物理属性参考 TurtleBot3 Burger，为 Gazebo 仿真做准备

- 模块化 xacro 文件组织是工程标准要求
 - 先通过 `check_urdf` 静态验证，再用 RViz 动态验证
-

讲次 3: 【Activity】移动机器人 URDF 练习解答与调试

知识点

本节提供完整的参考实现和系统的调试指南，帮助排查学员在练习中遇到的常见问题。

完整文件清单

```
my_robot_description/  
├─ CMakeLists.txt  
├─ package.xml  
├─ urdf/  
│   └─ my_robot.urdf.xacro  
│       └─ my_robot/  
│           ├── properties.xacro  
│           ├── materials.xacro  
│           ├── inertia_macros.xacro  
│           ├── macros.xacro  
│           ├── base.xacro  
│           ├── sensors.xacro  
│           ├── wheels.xacro  
│           └─ gazebo_materials.xacro  
├─ launch/  
│   └─ display.launch.py  
└─ rviz/  
    └─ my_robot.rviz
```

inertia_macros.xacro (完整版)

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:macro name="inertia_box" params="mass x y z">
    <inertia
      ixx="{mass / 12.0 * (y*y + z*z)}"
      ixy="0" ixz="0"
      iyy="{mass / 12.0 * (x*x + z*z)}"
      iyz="0"
      izz="{mass / 12.0 * (x*x + y*y)}"/>
    </xacro:macro>

  <xacro:macro name="inertia_cylinder_y" params="mass radius length">
    <inertia
      ixx="{mass / 12.0 * (3.0*radius*radius + length*length)}"
      ixy="0" ixz="0"
      iyy="{mass / 2.0 * radius*radius}"
      iyz="0"
      izz="{mass / 12.0 * (3.0*radius*radius + length*length)}"/>
    </xacro:macro>

  <xacro:macro name="inertia_cylinder_z" params="mass radius length">
    <inertia
      ixx="{mass / 12.0 * (3.0*radius*radius + length*length)}"
      ixy="0" ixz="0"
      iyy="{mass / 12.0 * (3.0*radius*radius + length*length)}"
      iyz="0"
      izz="{mass / 2.0 * radius*radius}"/>
    </xacro:macro>

</robot>
```

materials.xacro (完整版)

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <xacro:property name="color_blue" value="0.0 0.0 0.8 1.0"/>
  <xacro:property name="color_black" value="0.0 0.0 0.0 1.0"/>
  <xacro:property name="color_grey" value="0.5 0.5 0.5 1.0"/>
  <xacro:property name="color_dark_grey" value="0.2 0.2 0.2 1.0"/>

  <material name="blue">
    <color rgba="${color_blue}"/>
  </material>
  <material name="black">
    <color rgba="${color_black}"/>
  </material>
  <material name="grey">
    <color rgba="${color_grey}"/>
  </material>
  <material name="dark_grey">
    <color rgba="${color_dark_grey}"/>
  </material>

</robot>
```

常见问题排查

问题 1: RViz 中看不到机器人

```
# 检查 robot_description 是否发布
ros2 topic echo /robot_description --once | head -3

# 检查 Fixed Frame 是否正确
# RViz → Global Options → Fixed Frame = base_footprint
```

原因与修复:

原因	修复
robot_state_publisher 未启动	检查 Launch 文件
Fixed Frame 设为 <code>map</code> 或 <code>odom</code>	改为 <code>base_footprint</code>
xacro 展开失败	手动运行 <code>xacro file.xacro</code> 查看错误

问题 2: TF 树断裂

```
ros2 run tf2_tools view_frames
evince frames_*.pdf # Jazzy 生成带时间戳的 frames_YYYY-MM-DD_HH.MM.SS.pdf
```

现象	原因	修复
缺少 wheel link	joint 定义错误	检查 parent/child link 名称拼写
base_footprint 孤立	缺少 base_joint	添加 fixed joint
两个根节点	存在多个无 parent 的 link	确保只有 base_footprint 无 parent

问题 3：轮子在 RViz 中方向错误

正确：圆柱侧面朝外（轮子像一个圆盘）
 错误：圆柱顶面朝外（轮子像一个柱子）

修复：visual 和 collision 的 `<origin rpy="{pi/2} 0 0"/>` 不能遗漏。

问题 4：joint_state_publisher_gui 中看不到轮子关节

```
ros2 topic echo /joint_states --once
```

预期输出包含 `left_wheel_joint` 和 `right_wheel_joint`。如果缺失：- 检查 joint type 是否为 `continuous`（fixed 关节不会出现在 joint_states 中）- 重启 joint_state_publisher_gui

问题 5：xacro 展开报错

```
xacro my_robot.urdf.xacro 2>&1
```

错误	修复
No such file: my_robot/properties.xacro	检查 include 相对路径
property 'wheel_radius' undefined	检查 include 顺序，properties 必须最先
Invalid parameter "prefix"	宏调用缺少 prefix 参数

问题 6：check_urdf 报错

```
check_urdf /tmp/my_robot.urdf 2>&1
```

错误	修复
link 'xxx' is not unique	宏被调用了两次但 prefix 相同
joint 'xxx' has invalid parent	parent link 名称拼写错误
Could not parse inertial element	惯性值含非数字字符

调试工作流

```
xacro 展开 —失败—> 检查 include 路径和 property
|
成功
▼
check_urdf —失败—> 检查 link/joint 名称和惯性格式
|
成功
▼
ros2 launch —RViz 无模型—> 检查 robot_description 话题
|
有模型
▼
view_frames —树断裂—> 检查 joint parent/child
|
树完整
▼
拖动关节 —不旋转—> 检查 joint type 和 axis
|
正常
▼
✅ 里程碑 1 完成
```

关键点

- 调试顺序: xacro 展开 → check_urdf → Launch → TF 树 → 关节交互
 - 常见问题集中在 include 路径、joint 名称拼写和轮子 rpy 旋转
 - `ros2 topic echo` 和 `view_frames` 是最有效的诊断工具
 - 参考完整文件清单逐项对比自己的实现
 - 遇到问题时先隔离: 最小Launch (仅 robot_state_publisher) 排查
-

讲次 4: 交付：完整的移动机器人 URDF 文件

知识点

交付标准

项目二里程碑 1 的交付物为 `my_robot_description` 功能包，满足以下全部条件：

类别	标准
结构	6 link、5 joint, TF 树以 base_footprint 为根
物理	5 个实体 link 有 collision + inertial, 总质量 ~1 kg
模块化	≥5 个 xacro 子文件 + 1 个主文件
可视化	<code>ros2 launch my_robot_description display.launch.py</code> 正常显示
交互	<code>joint_state_publisher_gui</code> 可拖动轮子关节
仿真准备	Gazebo 材质和摩擦系数已配置

最终验证脚本

当前仓库中的 `my_robot_description/scripts/verify_urdf.sh` 如下：


```

#!/usr/bin/env bash
set -euo pipefail

echo "==== P2 milestone 1 URDF verification ====="

pkg_prefix="$(ros2 pkg prefix my_robot_description)"
xacro_file="${pkg_prefix}/share/my_robot_description/urdf/my_robot.urdf.xacro"
urdf_file="/tmp/my_robot_verify.urdf"
graph_dir="/tmp/my_robot_verify_graph"

echo "[1/6] Expand Xacro"
xacro "${xacro_file}" -o "${urdf_file}"

echo "[2/6] Check URDF"
check_urdf "${urdf_file}" >/tmp/my_robot_check_urdf.log
cat /tmp/my_robot_check_urdf.log

echo "[3/6] Count structure and physical tags"
link_count="$(grep -c '<link name' "${urdf_file}")"
joint_count="$(grep -c '<joint name' "${urdf_file}")"
collision_count="$(grep -c '<collision>' "${urdf_file}")"
inertial_count="$(grep -c '<inertial>' "${urdf_file}")"
gazebo_count="$(grep -c '<gazebo reference=' "${urdf_file}")"
echo "links=${link_count} joints=${joint_count} collisions=${collision_count} inertials=${inerti

test "${link_count}" -eq 6
test "${joint_count}" -eq 5
test "${collision_count}" -eq 5
test "${inertial_count}" -eq 5
test "${gazebo_count}" -ge 5

echo "[4/6] Generate URDF graph"
rm -rf "${graph_dir}"
mkdir -p "${graph_dir}"
(
  cd "${graph_dir}"
  urdf_to_graphviz "${urdf_file}" my_robot_verify >/dev/null
)
test -f "${graph_dir}/my_robot_verify.pdf"

echo "[5/6] Check launch and RViz assets"
test -f "${pkg_prefix}/share/my_robot_description/launch/display.launch.py"
test -f "${pkg_prefix}/share/my_robot_description/rviz/my_robot.rviz"

echo "[6/6] Check expected final frames"
grep -q 'back_caster_link' "${urdf_file}"
grep -q 'left_wheel_joint' "${urdf_file}"
grep -q 'right_wheel_joint' "${urdf_file}"

echo "==== Verification passed ====="

```

在 CMakeLists.txt 中添加脚本安装规则：

```
install(PROGRAMS scripts/verify_urdf.sh
  DESTINATION lib/${PROJECT_NAME}
)
```

```
chmod +x ~/ros2_ws/src/my_robot_description/scripts/verify_urdf.sh
cd ~/ros2_ws && colcon build --packages-select my_robot_description
source install/setup.bash
ros2 run my_robot_description verify_urdf.sh
```

交付物清单

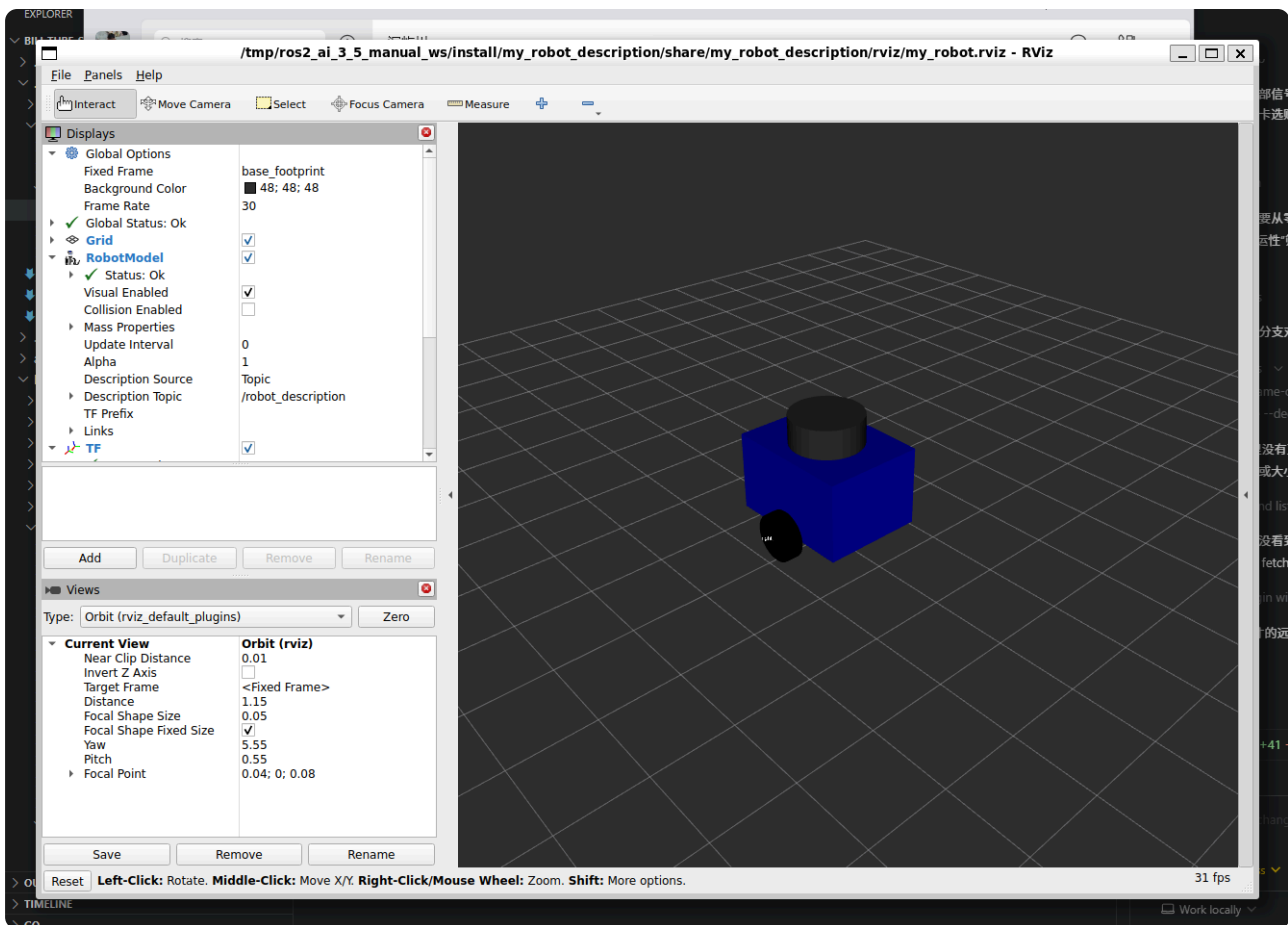
```
my_robot_description/      ← Git 仓库中的完整功能包
├─ urdf/
│   ├── my_robot.urdf.xacro ← 主入口
│   └─ my_robot/           ← 子模块
├─ launch/
│   └─ display.launch.py   ← 一键可视化
├─ rviz/
│   └─ my_robot.rviz       ← RViz 配置
├─ scripts/
│   └─ verify_urdf.sh      ← 验证脚本（CMakeLists 中 install）
├─ CMakeLists.txt
└─ package.xml
```

最终 TF 树

```
base_footprint
├─ base_link
│   ├── base_scan          (激光雷达)
│   ├── left_wheel_link    (左驱动轮, continuous)
│   ├── right_wheel_link   (右驱动轮, continuous)
│   └─ back_caster_link    (后万向轮, fixed)
```

物理参数汇总

Link	质量 (kg)	几何体	备注
base_link	0.826	box 0.178×0.138×0.109	蓝色底盘
base_scan	0.114	cylinder r=0.05 h=0.04	激光雷达
left_wheel_link	0.0285	cylinder r=0.033 h=0.018	驱动轮
right_wheel_link	0.0285	cylinder r=0.033 h=0.018	驱动轮
back_caster_link	0.005	sphere r=0.005	万向轮
合计	~1.002		接近 TurtleBot3 Burger



里程碑 1 完成标志

当你能成功执行以下全部命令时，里程碑 1 即告完成：

```
# 1. 构建
colcon build --packages-select my_robot_description

# 2. 验证 URDF
xacro $(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/my_robot.urdf.xacro | check_urdf /dev/stdin

# 3. 可视化
ros2 launch my_robot_description display.launch.py

# 4. 交互测试
# 在 joint_state_publisher_gui 中拖动轮子 → RViz 中轮子旋转

# 5. TF 树
ros2 run tf2_tools view_frames && evince frames_*.pdf
```

关键点

- 里程碑 1 交付物： `my_robot_description` 功能包，6 link / 5 joint
- 验证脚本可自动化检查 6 项核心指标
- 物理参数参考 TurtleBot3 Burger，总质量 ~1 kg

- 完成后机器人描述具备进入 Gazebo Harmonic 的基础，但尚未配置驱动控制和传感器插件
 - 第四阶段将在此 URDF 基础上启动 Gazebo 仿真
-

本节总结

里程碑 1 成果

第三阶段 3.1~3.5 学习路径

- 3.1 TF 坐标变换 → 理解坐标系
- 3.2 URDF 基础 → 创建 Link/Joint
- 3.3 Xacro 进阶 → 变量/宏/模块化
- 3.4 仿真准备 → Collision/Inertial/Gazebo
- 3.5 移动机器人 URDF ☒ → 完整可仿真模型

命令速查表

命令	说明
<code>ros2 launch my_robot_description display.launch.py</code>	可视化机器人
<code>xacro file.xacro \ check_urdf /dev/stdin</code>	一键校验
<code>ros2 run tf2_tools view_frames</code>	TF 树可视化
<code>ros2 topic echo /joint_states --once</code>	检查关节状态

下一步

恭喜完成项目二里程碑 1！你的移动机器人 URDF 已具备完整的结构描述和物理属性。

下一节进入 **第四阶段：仿真环境与硬件抽象**，你将：

- 在 **Gazebo Harmonic** 中加载并运行你的机器人模型
- 配置 **ros_gz_bridge** 实现 Gazebo 与 ROS 2 通信
- 添加差速驱动控制器，用键盘驱动机器人在仿真世界中运动
- 继续推进项目二里程碑 2：Gazebo 仿真 + 机械臂